

EvoRank: LLM-Guided Evolution of Multi-Objective Learning-to-Rank Pipelines

Rayhan Patel^{1,†} Shabaz Patel^{2,†}

¹University of Maryland, College Park ²Independent Researcher [†]equal contribution

GenAIECommerce'26: The Third Workshop on Agentic and Generative AI for E-Commerce, co-located with RecSys 2026, Minneapolis, MN, USA



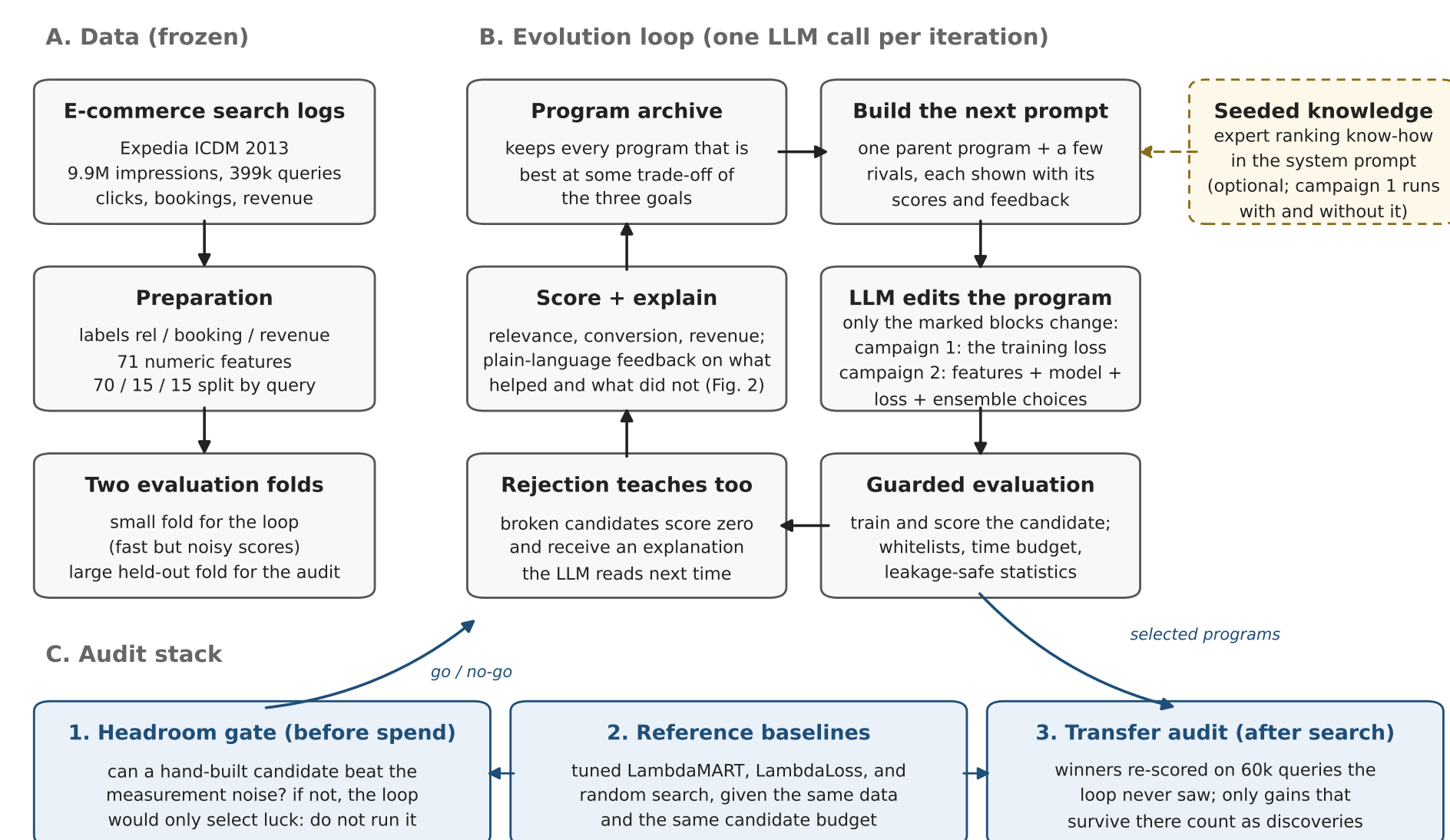
code + all results

The failure nobody reports

LLM-guided discovery loops (AlphaEvolve-style) report gains on the data the loop used to select candidates. Ranking evaluation is noisy by necessity, and selection under noise promotes lucky candidates: a loop can show steady progress while discovering nothing that transfers.

The claim. Whether the loop pays off is measurable before any LLM spend: the ratio of achievable effect sizes in the search space to the noise of the fitness signal. We package this as the headroom gate, and let a transfer audit on held-out queries, not the loop's own scores, decide what counts.

One loop, fully audited



One LLM call per iteration edits the marked blocks of a candidate program; guarded evaluation scores it on a small fitness fold; the audit lane decides what is real. Engine: SkyDiscover (AdaEvolve controller).

The audit stack (acceptance and stopping criteria, identical in every campaign):

- headroom gate: a hand-built candidate from the domain literature must beat the fitness fold's noise floor by **2.5×**
- paired query bootstrap on every comparison
- equal-budget random-search control
- equal Optuna tuning on both sides
- transfer audit: re-score every selected program on 60k held-out queries
- hypervolume + component-removal tests

Task. Expedia hotel search (ICDM 2013): 9.9M impressions, 399k queries, split 70/15/15 by query. Three competing objectives from real behavior: relevance (NDCG@10), conversion (booking NDCG@10), revenue (pooled dollar-weighted capture).

Campaign 1: selecting noise

Evolving the training objective (the LambdaMART gradient) looked great on the selection fold and failed the transfer audit, on 60k held-out queries:

- best evolved vs its own seed: **+0.0002** NDCG@10 ($p = 0.76$)
- blind random search transferred as well or better ($p = 0.03$)
- every method below tuned LambdaMART ($p < 0.001$)

Mechanism, measured: a single component's true effect is **0.0005–0.008** NDCG@10; the fold's measurement noise is **± 0.007 –0.009**. Picking the best score means picking luck.

Seeded domain knowledge made search faster, not truer: two of three seeded runs matched random search's entire budget within 1–3 iterations (unseeded: 7–23), but what transferred was unchanged. Gradient-level diagnostic feedback changed nothing; enlarging the evaluation fold (1.6k→6k queries) halved noise and tripled surviving gain, the only intervention that moved it.

Campaign 2: pipelines transfer

Same loop, wider space: features + model + loss + ensemble, with leakage-safe helpers (out-of-fold encodings). The gate passed first (**+0.013** $\geq 2.5\times$ noise). Three runs, 50 iterations, ~\$10 each:

	Val	Test	Test	Test
Method (fitness-fold trained)	NDCG@10	NDCG@10	book	revenue
Starting pipeline	0.4218	0.4222	0.4443	0.4105
LambdaMART + Optuna	0.4330	0.4296	0.4527	0.4231
Pipeline s0	0.4356	0.4316	0.4545	0.4257
Pipeline s1	0.4359	0.4322	0.4553	0.4215
Pipeline s2	0.4398	0.4352	0.4582	0.4239

Transfer audit on 59,902 held-out queries; bold = column best.

Best run: **+0.0057** NDCG@10 over the tuned baseline (95% CI [0.0041, 0.0073], $p < 0.0001$), **+0.0056** booking NDCG@10, revenue matching: significantly better relevance and conversion at matching revenue, in every run.

It holds at full scale

280k train / 60k test queries	NDCG@10	NDCG@38
Tuned baseline (small-fold tuning)	0.4349	0.4983
Tuned baseline (full-scale tuning)	0.4544	0.5132
Pipeline s1 (small-fold tuning)	0.4490	0.5080
Pipeline s1 (baseline's tuned settings)	0.4619	0.5185

Features alone: 0.4587 (+0.0043); the ensemble adds **+0.0032**. Full-scale bootstrap: **+0.0075** NDCG@10, $p < 0.0001$.

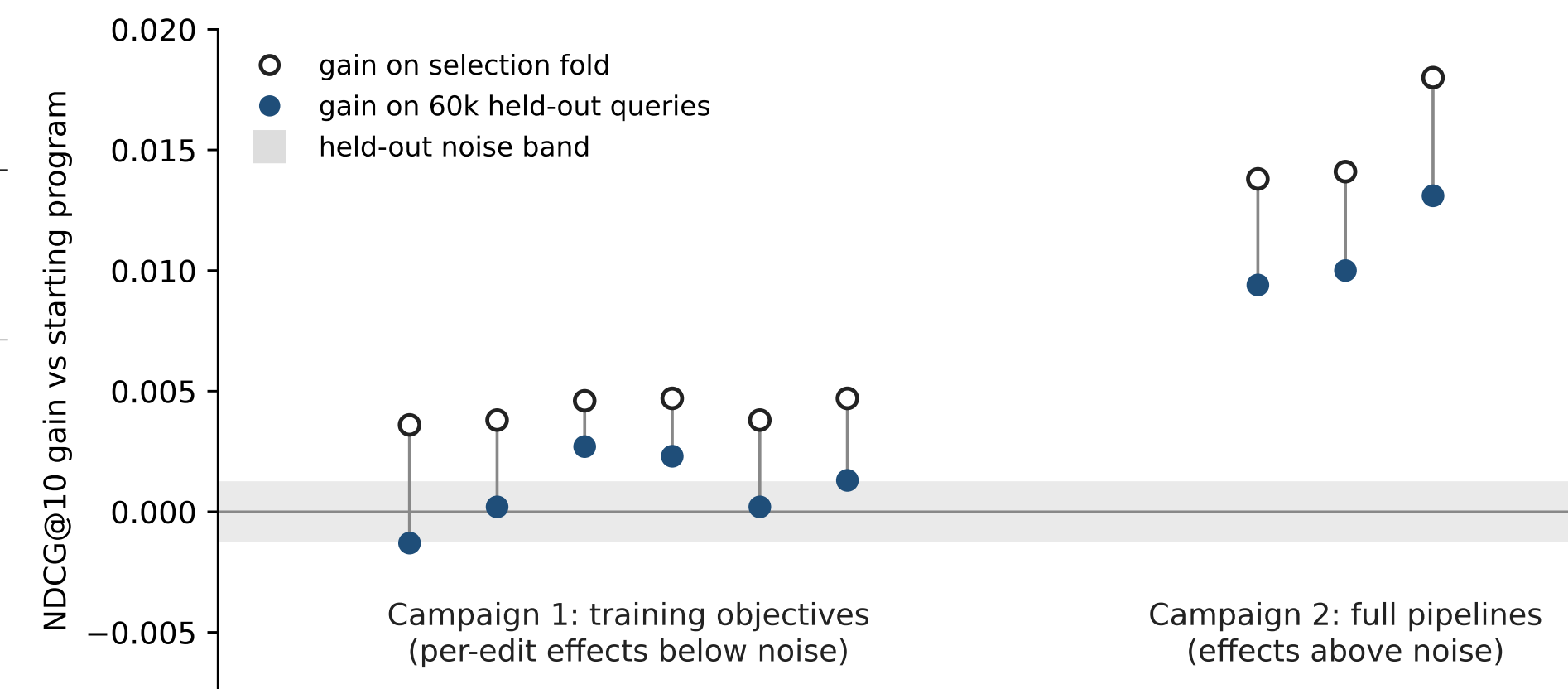
Kaggle late submission (one shot, hidden test set): discovered pipeline 0.5196 private NDCG@38 → would rank 20 of 340 (top 6%) in the 2013 challenge; tuned baseline 0.5140 (30th). Ten leaderboard places from the discovered structure alone.

What the search found

```
for c in [price, stars, review, loc1, loc2]:
    out[c+"_qrank"] = groupby(qid)[c].rank(pct=True)
out["prop_id_count"] = stats.count(df, "prop_id")
PIPELINE = {xgb:rankndcg + lgbm:lbmdarank +
             sklearn:ert,
             ensemble: zscore_weighted [0.5, 0.35, 0.15]}
```

The same readable core in all three runs: within-query percentile ranks, count encodings, a lean feature set, and a diverse ensemble under per-query z-scores: the ICDM 2013 winners' essential structure, rediscovered and extended autonomously.

The study in one picture



Per-run NDCG@10 gains over the starting program, on the selection fold (open) vs 60k held-out queries (filled). Campaign 1 collapses into the noise band; campaign 2 survives.

Open everything: code, configs, seeds, logs, discovered programs, audit scripts at github.com/shabazpatel/evorank. Limitations: one dataset (the only public LTR set with conversion and revenue labels); three runs per condition; revenue is a proxy; audited for transfer, not online effects.

Use this on your ranking stack

1. Measure noise

Paired query bootstrap on the evaluation fold you can afford: that half-width is your noise floor.

2. Hand-build one candidate

One reference candidate from your domain's literature, before any LLM spend.

3. Gate the space

Require its gain $\geq 2.5\times$ the noise floor. No headroom, no loop: keep your budget.

4. Run guarded

Leakage-safe statistics, whitelists, budgets, and rejection messages the model learns from.

5. Audit, then believe

A held-out transfer audit, never the loop's own scores, decides what counts.